

JUNIT ATIVIDADE: MANIPULAÇÃO DE STRINGS

1. Crie uma classe de testes para uma classe de Manipulação de Strings capaz de realizar as seguintes operações:
 - Inverter uma string;
 - Contar o número de ocorrências de um caractere;
 - Verificar se uma string é um palíndromo;
 - Converter uma string para maiúsculas e minúsculas.

```
public class StringManager { no usages
    public String inverter(String entrada) { no usages
        if (entrada == null) return null;
        return new StringBuilder(entrada).reverse().toString();
    }

    public int contarOcorrencias(String entrada, char alvo) { no usages
        if (entrada == null) return 0;
        int contador = 0;
        for (char c : entrada.toCharArray()) {
            if (c == alvo) contador++;
        }
        return contador;
    }

    public boolean ePalindromo(String entrada) { no usages
        if (entrada == null) return false;
        String limpo = entrada.replaceAll(regex: "\\s+", replacement: "").toLowerCase();
        String invertido = new StringBuilder(limpo).reverse().toString();
        return limpo.equals(invertido) && !limpo.isEmpty();
    }
}
```

```
    public String paraMaiusculas(String entrada) { no usages
        return (entrada == null) ? null : entrada.toUpperCase();
    }

    public String paraMinusculas(String entrada) { no usages
        return (entrada == null) ? null : entrada.toLowerCase();
    }
}
```

2. Considere testar os cenários a seguir:
 - Strings vazias, nulas e com diferentes tipos de caracteres;
 - Strings com caracteres especiais;

- Strings curtas e strings longas.

```
public class StringManagerTest {
    private StringManager gerenciador; 19 usages

    @BeforeEach
    void setup() {
        gerenciador = new StringManager();
    }

    @Test
    @DisplayName("Deve inverter strings normais, curtas e com caracteres especiais")
    void testarInverter() {
        assertEquals(expected: "java", gerenciador.inverter(entrada: "avaj"));
        assertEquals(expected: "!@#", gerenciador.inverter(entrada: "#@!"));
        assertEquals(expected: "a", gerenciador.inverter(entrada: "a"));
        assertEquals(expected: "", gerenciador.inverter(entrada: ""));
        assertNull(gerenciador.inverter(entrada: null));
    }
}
```

```
@Test
@DisplayName("Deve contar ocorrências de caracteres corretamente")
void testarContarOcorrencias() {
    assertEquals(expected: 3, gerenciador.contarOcorrencias(entrada: "banana", alvo: 'a'));
    assertEquals(expected: 1, gerenciador.contarOcorrencias(entrada: "Teste de Caractere Especial!", alvo: 'e'));
    assertEquals(expected: 0, gerenciador.contarOcorrencias(entrada: "", alvo: 'z'));
    assertEquals(expected: 0, gerenciador.contarOcorrencias(entrada: null, alvo: 'a'));
}

@ParameterizedTest
@ValueSource(strings = {"arara", "Ana", "A base do teto desaba", "Racecar"})
@DisplayName("Deve validar palíndromos válidos (ignorando case e espaços)")
void testarEPalindromoValido(String entrada) {
    assertTrue(gerenciador.ePalindromo(entrada));
}

@Test
@DisplayName("Deve validar cenários inválidos de palíndromos")
void testarEPalindromoInvalido() {
    assertFalse(gerenciador.ePalindromo(entrada: "java"));
    assertFalse(gerenciador.ePalindromo(entrada: ""));
    assertFalse(gerenciador.ePalindromo(entrada: null));
}
}
```

```
@Test
@DisplayName("Deve converter case corretamente incluindo strings longas")
void testarConversaoDeCaixa() {
    String longa = "ESTA É UMA STRING REALMENTE MUITO LONGA PARA TESTAR O COMPORTAMENTO DO SISTEMA";
    assertEquals(expected: "JAVA", gerenciador.paraMaiusculas(entrada: "java"));
    assertEquals(expected: "teste", gerenciador.paraMinusculas(entrada: "TESTE"));
    assertEquals(longa.toLowerCase(), gerenciador.paraMinusculas(longa));
    assertNull(gerenciador.paraMaiusculas(entrada: null));
    assertNull(gerenciador.paraMinusculas(entrada: null));
}
}
```